

ModuleDrivenFramwork Project Guide

1. Project Summary

ModuleDrivenFramwork is an ASP.NET Core Web API built on .NET 10 with a modular feature layout. The application starts from a single host and loads feature modules dynamically based on configuration.

Current modules in this repository:

- Auth: implemented and enabled by default
- Payment: implemented but disabled by default

The project already includes:

- JWT authentication
- Swagger for API exploration in development
- Module-aware controller loading
- Module-aware endpoint hiding in Swagger
- Entity Framework Core support for Auth with SQLite or MySQL

2. Technology Stack

- .NET 10 Web API
- ASP.NET Core Controllers
- Swagger via Swashbuckle
- JWT Bearer authentication
- Entity Framework Core
- SQLite and MySQL support for the Auth module

Important packages are declared in ModuleDrivenFramwork.csproj:

- Microsoft.AspNetCore.Authentication.JwtBearer
- Microsoft.EntityFrameworkCore.Design
- Microsoft.EntityFrameworkCore.Sqlite
- Pomelo.EntityFrameworkCore.MySql
- Swashbuckle.AspNetCore

3. Solution Structure

Main folders:

- Program.cs: application bootstrap and middleware pipeline
- Common/Auth: current-user abstractions
- Common/Modules: module registration, module filtering, and module configuration helpers
- Controllers: non-module controllers such as WeatherForecast
- Modules/Auth: authentication and authorization module
- Modules/Payment: payment demo module
- docs: project documentation
- scripts: helper scripts

Module layout pattern:

```
Modules/<ModuleName>/
  <ModuleName>ModuleExtensions.cs
Application/
  DTOs/
  Interfaces/
Controllers/
Domain/
  Entities/
Persistence/
  Migrations/
Services/
```

4. How the Module System Works

The module system is implemented in Common/Modules.

Behavior:

- All concrete classes implementing IAppModule are discovered at startup.
- Each module is checked against configuration under Modules:<ModuleName>:Enabled.
- Disabled modules are not registered into dependency injection.
- Disabled module controllers are excluded from MVC controller discovery.
- Disabled module endpoints are removed from Swagger.
- A protection filter also returns 404 if a disabled module controller is somehow reached.

This makes modules operationally switchable through configuration without removing code from the project.

5. Runtime Configuration

The main runtime configuration is in appsettings.json.

Current important settings:

- ConnectionStrings: database connections
- Modules: enables or disables modules
- Jwt: signing key and token expiry
- Logging

Current module configuration behavior:

- Auth is enabled
- Payment is disabled

Important note:

- Replace development secrets before any production deployment.
- Do not keep real database passwords or long-term JWT secrets in source control.

6. Running the Project

Prerequisites

- .NET SDK 10 installed
- A reachable database for the provider configured for Auth

Default development URLs

From launchSettings.json:

- HTTP: http://localhost:5019
- HTTPS: https://localhost:7117

Start the API

From the ModuleDrivenFramework folder:

```
dotnet restore
dotnet run
```

Swagger opens in development at:

```
http://localhost:5019/swagger
```

or

```
https://localhost:7117/swagger
```

7. Authentication Setup

JWT authentication is configured in Program.cs.

What happens during startup:

- The JWT key is loaded from configuration
- Bearer authentication is registered
- Issuer signing key validation is enabled
- Issuer and audience validation are currently disabled
- Authorization middleware is enabled

For production hardening, consider:

- moving the JWT secret to environment variables or a secret store
- enabling issuer and audience validation
- rotating secrets regularly

8. Database Behavior

The Auth module supports two providers:

- sqlite
- mysql

Provider resolution rules:

- If Modules:Auth:Database:Provider is set, that value is used.
- Otherwise the provider is inferred from the connection string.
- If nothing is configured, the fallback is SQLite using Data/modulerdrevenframework.db.

Startup behavior:

- AuthDbContext is registered during module registration.
- The Auth module runs database migrations automatically during initialization.

9. Implemented Modules

9.1 Auth Module

Purpose:

- user registration and login
- token refresh
- user management
- role management
- permission management

Persistence:

- Entity Framework Core DbContext
- seeded roles and permissions
- automatic migration on startup

Seeded data includes:

- roles: User, Admin
- permissions:
 - auth.profile.read - auth.profile.update - auth.tokens.refresh - auth.users.manage

Implemented endpoints:

```
POST  /api/auth/login
POST  /api/auth/register
POST  /api/auth/refresh

GET    /api/users
GET    /api/users/{id}
POST   /api/users
PUT    /api/users/{id}
DELETE /api/users/{id}
```

```
GET    /api/roles
GET    /api/roles/{id}
POST   /api/roles
PUT    /api/roles/{id}
DELETE /api/roles/{id}

GET    /api/permissions
GET    /api/permissions/{id}
POST   /api/permissions
PUT    /api/permissions/{id}
DELETE /api/permissions/{id}
```

9.2 Payment Module

Purpose:

- demo payment charge and refund workflow

Current status:

- implemented in memory only
- disabled by default in appsettings.json
- no persistent database storage

Endpoints when enabled:

```
GET    /api/payments
GET    /api/payments/{id}
POST   /api/payments/charge
POST   /api/payments/{id}/refund
```

10. How to Use the API

Recommended manual workflow:

1. Start the project with dotnet run.
2. Open Swagger.
3. Register a user through /api/auth/register.
4. Log in through /api/auth/login.
5. Save the access token and refresh token.
6. Use management endpoints for users, roles, and permissions.
7. Refresh the token through /api/auth/refresh when needed.

Example register request:

```
{
  "firstName": "Admin",
  "lastName": "User",
  "email": "admin@example.com",
  "password": "P@ssw0rd123"
}
```

Example login request:

```
{
  "email": "admin@example.com",
  "password": "P@ssw0rd123"
}
```

Example refresh request:

```
{
  "refreshToken": "paste-refresh-token-here"
}
```

To call authenticated endpoints from a client:

```
Authorization: Bearer <access-token>
```

11. Module Enable or Disable

Modules are controlled from appsettings.json.

Example:

```
"Modules": {
  "Auth": {
    "Enabled": true,
    "Database": {
      "Provider": "mysql",
      "ConnectionStringName": "Repository"
    }
  },
  "Payment": {
    "Enabled": false
  }
}
```

When a module is disabled:

- its services are not registered
- its controllers are not loaded
- its endpoints are removed from Swagger
- direct requests return 404

12. Fixed Module Scaffold Command

The original ad hoc command creates the folder tree and empty files, but it is hard to reuse and does not validate input.

Original command:

```
MODULE=Inventory && ROOT="Modules/$MODULE" && mkdir -p
"$ROOT"/{Application/DTOs,Application/Interfaces,Controllers,Domain/Entities,Persistence/Migrations,Services}
&& touch "$ROOT/${MODULE}ModuleExtensions.cs" "$ROOT/Application/DTOs/${MODULE}Dtos.cs"
"$ROOT/Application/Interfaces/I${MODULE}Service.cs" "$ROOT/Controllers/${MODULE}Controller.cs"
"$ROOT/Domain/Entities/${MODULE}Entity.cs" "$ROOT/Persistence/${MODULE}DbContext.cs"
"$ROOT/Persistence/${MODULE}DatabaseConfiguration.cs" "$ROOT/Persistence/${MODULE}DbContextFactory.cs"
"$ROOT/Persistence/${MODULE}SeedData.cs" "$ROOT/Services/${MODULE}Service.cs"
```

Preferred fixed usage:

```
./scripts/scaffold-module.sh Inventory
```

What the fixed script improves:

- validates the module name argument
- works for any module name without editing the command
- creates the same structure consistently
- avoids repeating a long shell one-liner in documentation or terminal history

If you still want a one-line version, use this cleaner form from the ModuleDrivenFramework folder:

```
MODULE="Inventory"; ROOT="Modules/$MODULE"; mkdir -p
"$ROOT"/{Application/DTOs,Application/Interfaces,Controllers,Domain/Entities,Persistence/Migrations,Services};
touch "$ROOT/${MODULE}ModuleExtensions.cs" "$ROOT/Application/DTOs/${MODULE}Dtos.cs"
"$ROOT/Application/Interfaces/I${MODULE}Service.cs" "$ROOT/Controllers/${MODULE}Controller.cs"
"$ROOT/Domain/Entities/${MODULE}Entity.cs" "$ROOT/Persistence/${MODULE}DbContext.cs"
"$ROOT/Persistence/${MODULE}DatabaseConfiguration.cs" "$ROOT/Persistence/${MODULE}DbContextFactory.cs"
"$ROOT/Persistence/${MODULE}SeedData.cs" "$ROOT/Services/${MODULE}Service.cs"
```

13. Adding a New Module

If you want to add another module later, use this sequence:

1. Run `./scripts/scaffold-module.sh` with the new module name.
2. Define DTOs and service interfaces.
3. Implement controllers and services.
4. Add persistence if the module needs a database.
5. Add configuration under `Modules:<ModuleName>`.
6. Add tests for service and controller behavior.

14. Known Gaps

- There is no consolidated README for the solution root yet.
- Payment uses in-memory storage only.
- Auth controller actions do not yet expose structured validation or authorization policies for each management endpoint.
- Production secrets handling still needs tightening.

15. Document Output

This Markdown file is the source used for the PDF version of the project guide.

Generated output:

- `docs/PROJECT_GUIDE.md`
- `docs/PROJECT_GUIDE.pdf`